WCRC 2026 · 2차 교육

Mission2 → Mission2B 변경 내용

Dropbox B 추가, 지그재그 탐색, 새로운 종료 경로까지
무엇이 왜 달라졌는지 함께 살펴봅니다



Mission2 수업을 들은 참가 학생용

오늘 수업이 끝나면 할 수 있는 것

1

설명한다

Mission2B에서 달라진 점 6가지를 이유와 함께 말할 수 있습니다.

2

찾는다

달라진 동작을 코드에서 함수 이름으로 찾아 읽을 수 있습니다.

3

맞춘다

내 로봇에 필요한 자세와 설정값을 확인하고 조정할 수 있습니다.

오늘의 순서

복습: 그대로인 뼈대 → 달라진 점 6가지 → 내 로봇 준비 → 확인 실습

복습

기본 흐름은 Mission2 그대로입니다



그대로인 것

- 블록 하나를 옮긴 뒤에는 **항상 CELL1부터** 다시 찾습니다.
- 카메라는 **색과 좌우 위치**를, PSD는 **벽·박스까지 거리**를 맞춥니다.
- 팔 자세는 **EEPROM에 저장한 자세**를 번호로 불러 씁니다.

달라진 점 6가지

1 목적지가 16칸

Dropbox B가 추가되어 A와 B 각 8칸, 어디든 넣을 수 있습니다.

2 미션 순서 읽기 강화

최대 9번 촬영하고, 일부만 읽혀도 읽은 블록만큼은 수행합니다.

3 지그재그 층 탐색

홀수 열은 위, 짝수 열은 아래에서 시작해 팔 움직임을 줄입니다.

4 제자리 90° 회전

B로 가기 위해 로봇이 처음으로 제자리에서 회전합니다.

5 종료 경로 3가지

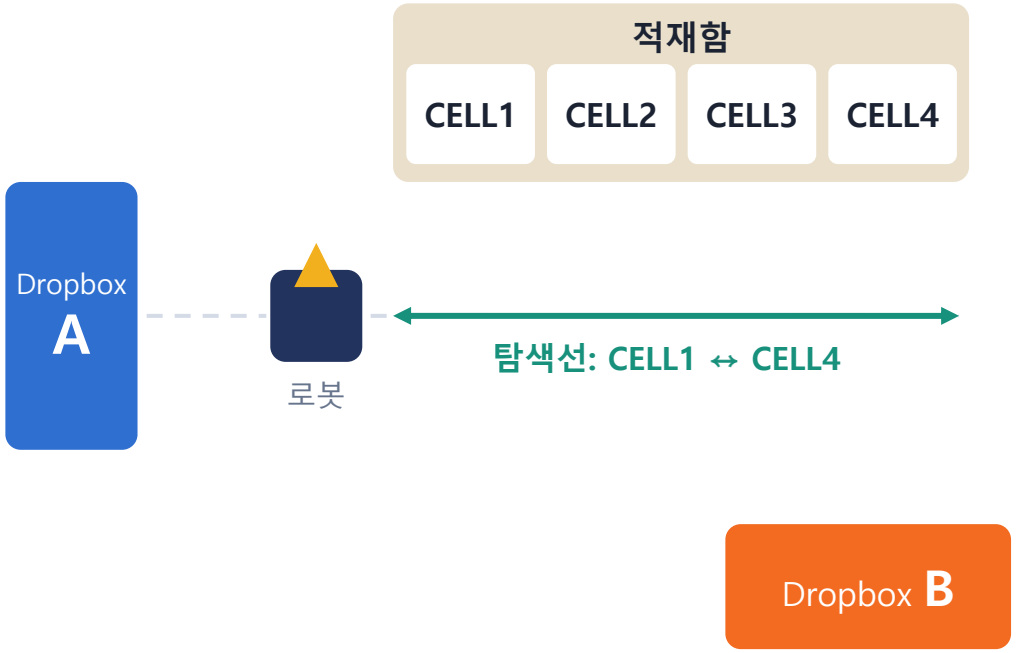
마지막 블록을 어디에 넣었는지에 따라 끝나는 길이 달라집니다.

6 안전 검사와 설정값

출발 전에 자세를 점검하고, 이동 속도와 대기 시간을 다듬었습니다.

전체 그림

로봇이 움직이는 곳을 먼저 봅니다



코드의 이동 방향과 거리로 그린 개념도입니다. 실제 크기 비율과는 다릅니다.

적재함

로봇 정면. 4열(CELL1~4) × 3층(L1~L3) 12칸에서 블록을 찾습니다.

Dropbox A

로봇 왼쪽. 팔을 왼쪽으로 돌려 넣습니다. (Mission2와 같음)

Dropbox B

로봇 뒤쪽. 뒤돌아 정면으로 다가가 넣습니다. (새로 추가)

종료 Zone

출발한 쪽. 블록을 모두 옮기면 여기로 돌아와 멈춥니다.

변경 1 · 목적지 16칸

배정표에서 B 칸은 '8 + 칸 번호'로 적습니다

```
// { 색상, 낙하 칸 }   블록 하나에 한 줄
const MissionBlockAssignment
      missionAssignments[] = {
  { 1, 1   },   // 첫 번째 빨강 → A1
  { 1, 8+5 },   // 두 번째 빨강 → B5
  { 3, 4   },   // 노랑 → A4
  { 4, 8+2 },   // 첫 번째 초록 → B2
  { 4, 5   },   // 두 번째 초록 → A5
  { 5, 6   }    // 파랑 → A6
};
```

색상 번호

1 빨강 2 주황 3 노랑 4 초록 5 파랑 6 보라

- 같은 색은 **최대 2줄**, 블록은 모두 **최대 8개**
- 줄 순서는 운반 순서가 아닙니다. **순서는 카메라가 읽습니다.**

배정표 번호가 낙하 자세 번호로 바뀝니다

Dropbox A

1	2	3	4	5	6	7	8

배정표에 적는 값: **1 ~ 8**

EEPROM 낙하 자세: **11 ~ 18** (10 + 칸 번호)

Dropbox B

9	10	11	12	13	14	15	16

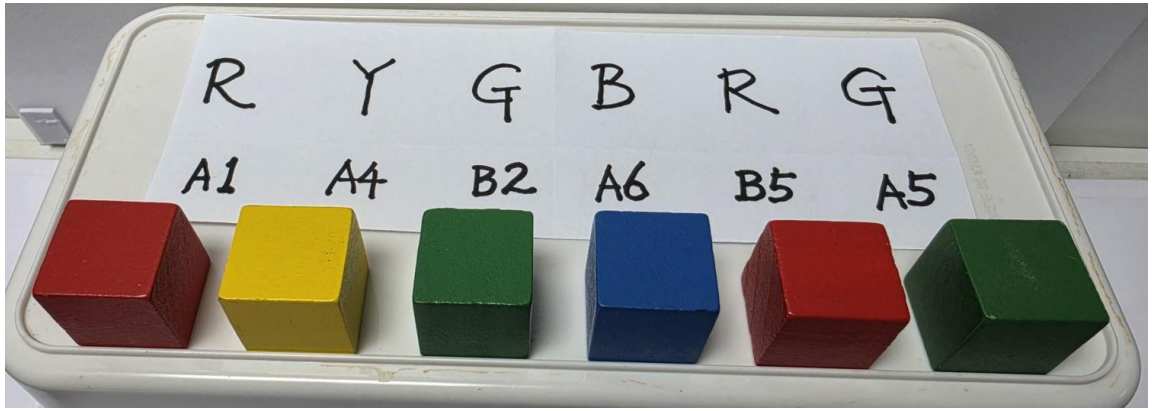
배정표에 적는 값: **8+1 ~ 8+8** (= 9 ~ 16)

EEPROM 낙하 자세: **19 ~ 26** (18 + 칸 번호)

같은 칸을 두 번 적으면 출발 전에 오류로 멈춥니다.

변경 1 · 목적지 16칸

카메라가 읽은 순서대로 목적지가 정해집니다



미션지시 블록 예시. 로봇은 왼쪽부터 차례로 읽습니다.

읽은 순서	색상	목적지	
1	빨강	A1	
2	노랑	A4	
3	초록	B2	
4	파랑	A6	
5	빨강	B5	두 번째 빨강
6	초록	A5	두 번째 초록

같은 색이 두 번 나오면?

배정표에서 그 색의 **아직 쓰지 않은 첫 줄**을 차례로 씁니다.
그래서 첫 번째 빨강은 A1, 두 번째 빨강은 B5가 됩니다.

```
SetMissionBlockTargets()
```

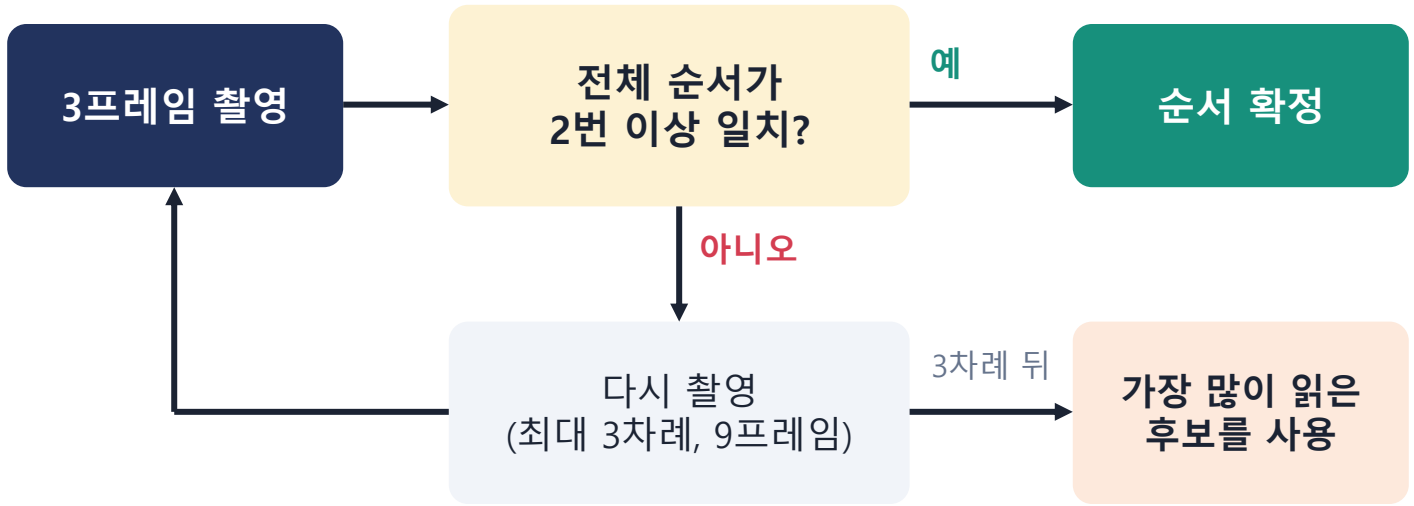
한 번 더 보고, 읽은 만큼은 수행합니다

Mission2

- 3프레임 촬영
- 전체 순서가 2번 일치해야 인정
- 실패 시 **배정표 줄 순서**로 진행

지시 순서와 다르게 옮길 수 있었습니다

Mission2B



쓸 수 있는 후보가 하나도 없으면 **오류 정지**합니다. 일부만 읽혔다면(예: 6개 중 4개) **읽은 4개만** 옮깁니다.

한 자세로 모두 읽고, 붙은 블록도 구분합니다



인식 자세에서 카메라가 본 화면 (s = 색상 번호)

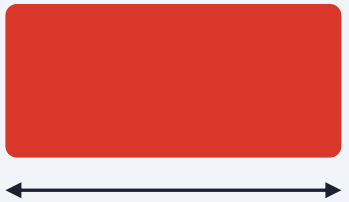
새로 쓰는 자세 2개

34 `START_POSE` 출발 대기 자세

27 `FIND_MISSION_1SHOT` 한 번에 읽는 자세

장애물 앞 보정 뒤, 왼쪽으로 가면서 27번 자세를 잡습니다.

붙어 있는 같은 색 블록



폭 72px 이상 → **2개**



따로 보이면 → **각각 1개**

2개로 세는 조건 (셋 모두 맞아야 합니다)

1. 폭이 **72px 이상** (블록 1개 폭 45px × 1.6)
2. 그 색이 배정표에 **2개** 있다
3. 화면에 그 색 덩어리가 **하나뿐**이다

Mission2는 ①만 확인했습니다. ②와 ③이 새로 추가된 조건입니다.

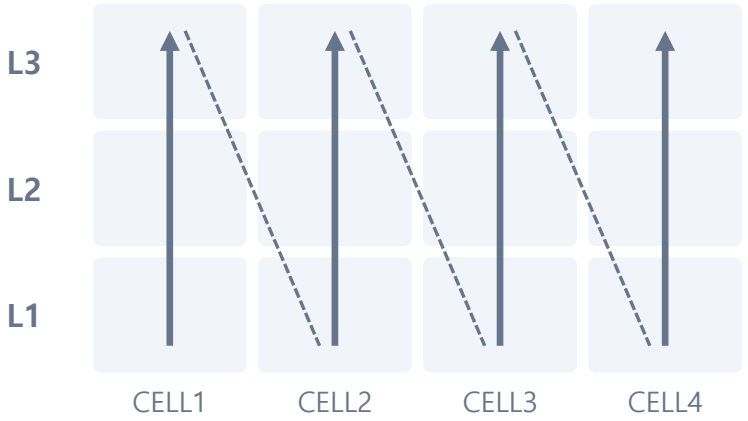
변경 3 · 적재함 탐색

홀수 열은 위에서, 짝수 열은 아래에서 시작합니다

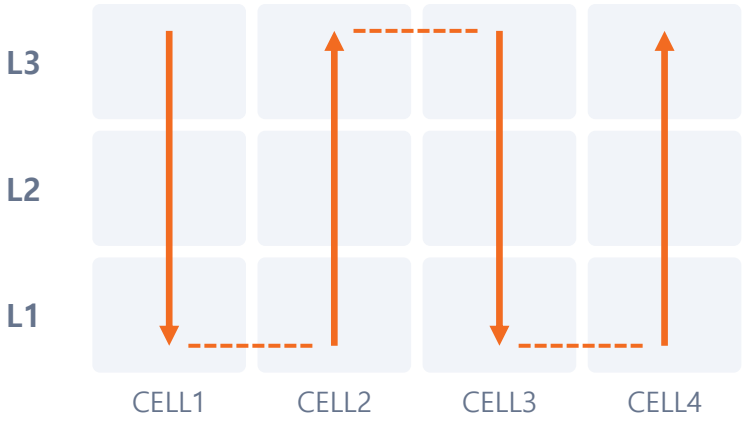


실제 적재함: 4열 × 3층 (아래부터 L1, L2, L3)

Mission2: 모든 열을 아래부터



Mission2B: 지그재그



점선: 열을 옮길 때 팔이 움직이는 거리

이웃 열은 같은 층에서 시작

앞 열의 마지막 층과 같은 자세라서 팔 명령(350ms)을 건너뜁니다.

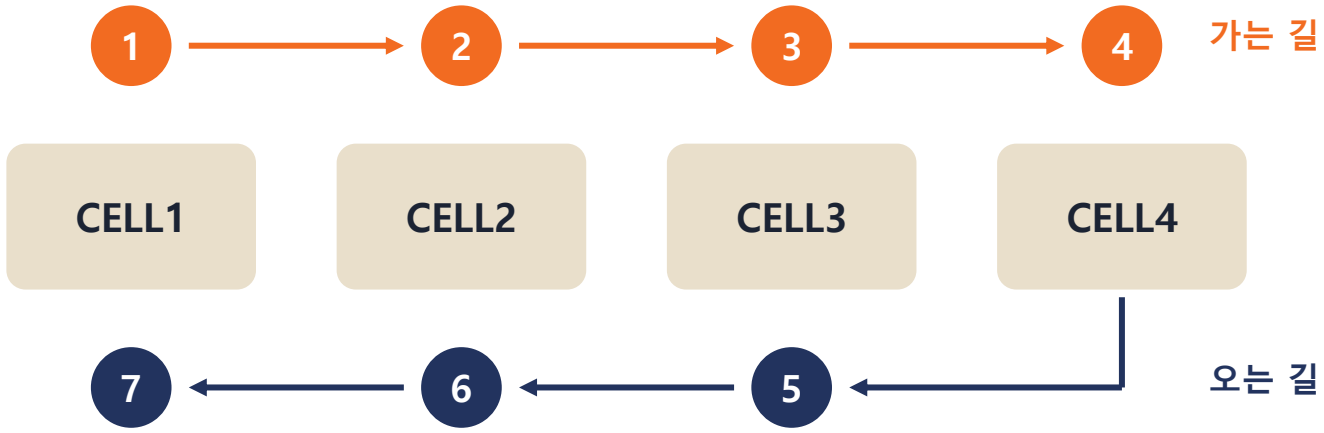
촬영 전 대기 1,000 → 80ms

층 자세를 잡은 뒤 기다리는 시간이 크게 줄었습니다.

열 번호의 홀짝으로 결정

`columnNo % 2 == 1` 이면 위에서 아래로, 아니면 아래에서 위로.

한 번 왕복하고, 마지막 CELL1까지 확인합니다



Mission2

1 → 2 → 3 → 4 → 3 → 2

이 순서를 **최대 2번** 반복

Mission2B

1 → 2 → 3 → 4 → 3 → 2 → 1

한 번만 왕복 (`SEARCH_SWEEP_LIMIT` 2 → 1)

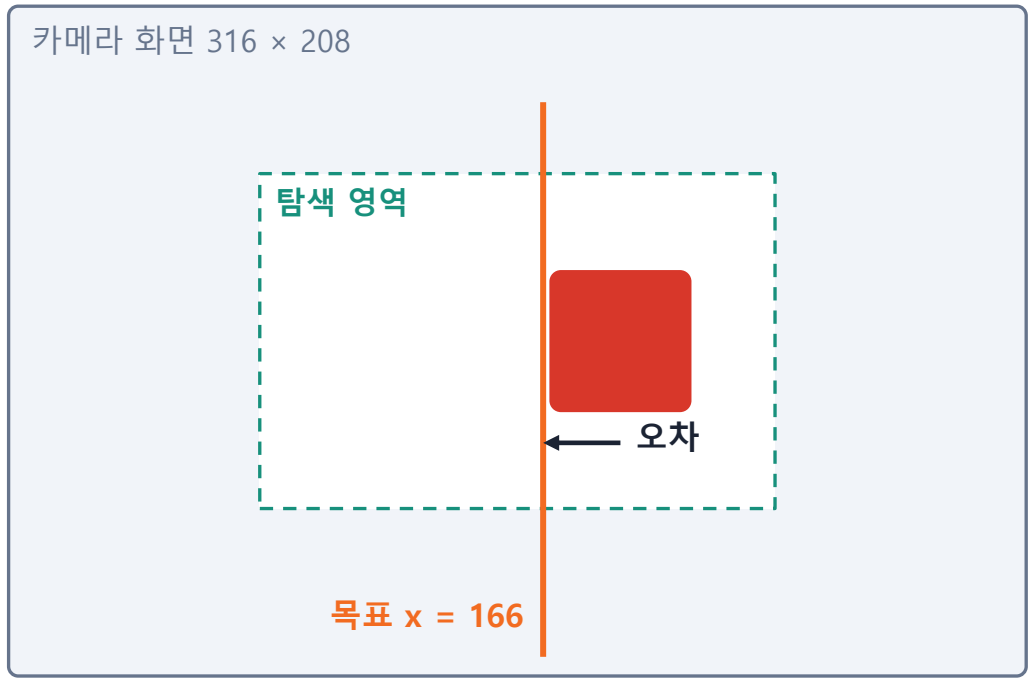
끝까지 못 찾으면

'삑' 세 번 뒤 다음 색으로 넘어가 CELL1부터 다시 찾습니다. 남은 색이 없으면 종료 경로로 갑니다.

칸 앞에서 거리 맞추기

가는 길 CELL1·2·3, 오는 길 CELL3·2에서 탐색 전에 전방 PSD(FR 315)로 적재함과의 거리를 맞춥니다.

집기 전 좌우 맞춤은 더 엄격해졌습니다



예) x = 190이면 (190 − 166) × 0.30 = **7.2mm** 옆으로 이동

1 **안정적으로 찾기**
5번 찍어 3번 이상 보이면 x의 중앙값을 씁니다.

2 **오차만큼 옆으로**
허용 범위 밖이면 (x − 166) × 0.30mm 만큼 이동합니다.

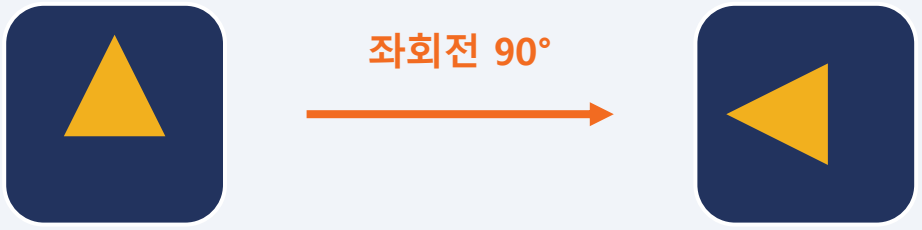
3 **다시 확인**
맞으면 집고, 아니면 돌아가서 다음 층을 봅니다.

±2mm

허용 오차
Mission2의 ±3mm에서 줄었습니다.
더 정확히 맞아야 집습니다.

로봇이 처음으로 제자리에서 회전합니다

```
// RotateXxx90Degree( dxl, 속도, 추가각 )
RotateLeft90Degree ( dxl, MOVE_MM_PER_SEC, 0.0f );
RotateRight90Degree( dxl, MOVE_MM_PER_SEC, 1.5f );
// 속도 = 160mm/s,  추가각 = 90°에 더하는 각도(도)
```



회전 전 회전 후

네 바퀴를 왼쪽·오른쪽 반대 방향으로 같은 거리만큼 굴립니다. 노란 삼각형이 로봇의 앞입니다.

247.4mm

90° 도는 데 필요한 바퀴 이동 거리

(135 + 180) ÷ 2 × π/2

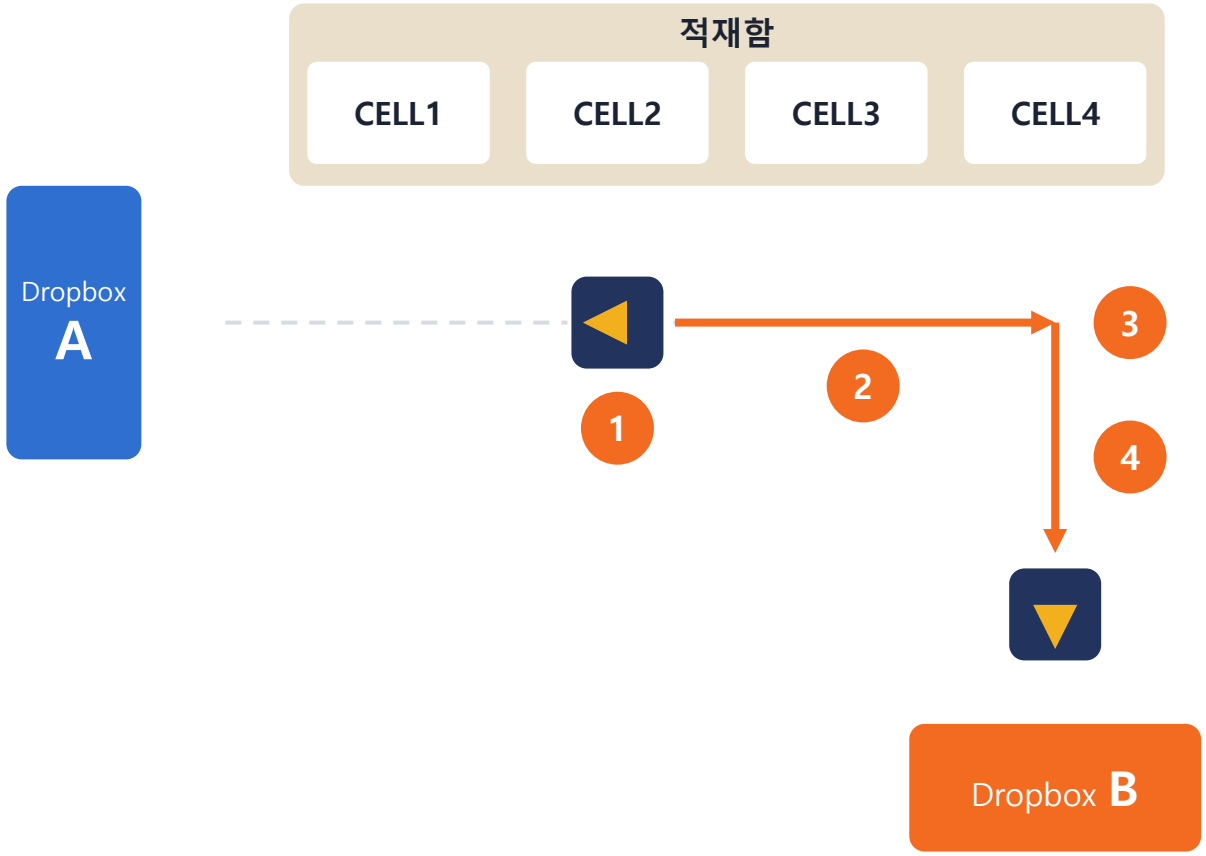
약 1.5초

160mm/s로 한 번 회전하는 시간

각도 센서는 없습니다

바퀴를 굴린 거리만 믿고 돕니다. 바닥과 배터리에 따라 조금 덜 돌거나 더 돌 수 있어서, 구간마다 '추가각'으로 맞춥니다.

B로 가는 길: 좌회전, 후진, 좌회전, 전진



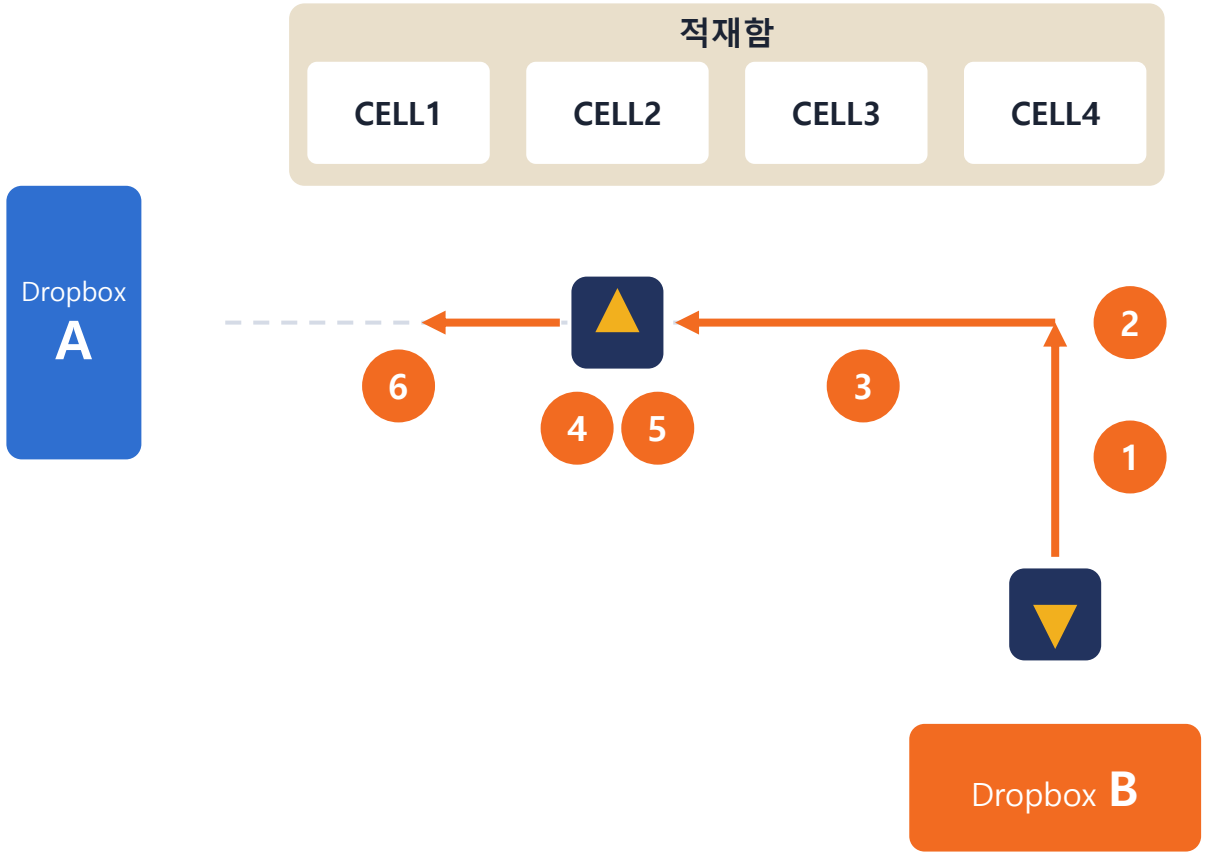
CELL2에서 집은 경우

1. **좌회전 90°**
   Dropbox A 쪽(왼쪽)을 바라봅니다.
2. **후진**
   등 쪽이 CELL4 방향입니다. 거리는 아래 표와 같습니다.
3. **좌회전 90°**
   이제 Dropbox B를 정면으로 봅니다.
4. **전진 366mm**
   B 앞에 도착해서 낙하 자세를 잡습니다.

집은 열	CELL1	CELL2	CELL3	CELL4
② 후진 거리	193mm	132mm	64mm	없음

```
GotoDropboxB( columnNo )
```

B에서 돌아올 때는 CELL2를 거쳐 CELL1으로



1	**후진 366mm**	B 앞에서 물러납니다.
2	**우회전 90° (+1.0°)**	Dropbox A 쪽을 봅니다.
3	**전진 124mm**	CELL2 앞까지 갑니다.
4	**우회전 90° (+1.5°)**	적재함을 다시 정면으로 봅니다.
5	**탐색 자세 준비**	첫 탐색 층(L3) 자세를 잡습니다.
6	**좌진 67mm**	CELL1에서 다음 블록을 찾습니다.

추가각 +1.0°, +1.5°는 제공 코드에서 맞춘 값입니다. 내 로봇은 직접 돌려 보고 정합니다.

`ReturnFromDropboxBToCell1()`

변경 4 · Dropbox B 경로

A와 B는 넣는 방법이 다릅니다

Dropbox A에 넣을 때

1. 팔을 **왼쪽(–90°)** 으로 돌립니다.
2. 왼쪽으로 이동합니다. CELL1부터 차례로 **124, 191, 259, 323mm**
3. 낙하 자세(500ms)를 잡고 그리퍼를 엽니다.
4. 팔을 접고 오른쪽으로 124mm 가면 CELL1입니다.

Dropbox B에 넣을 때

1. 팔을 **접은 채** 좌회전, 후진, 좌회전, 전진 366mm
2. 낙하 자세(500ms)를 잡고 그리퍼를 엽니다.
3. 팔을 접습니다.
4. CELL2를 거쳐 CELL1으로 돌아옵니다. 마지막 블록이면 바로 종료합니다.

```
const bool useDropboxB = destination > DROPBOX_CELL_COUNT;    // 목적지가 9 이상이면 B
```

마지막 블록을 넣은 곳에 따라 끝나는 길이 다릅니다



마지막 블록을 A에 넣었다

A 옆에서 그대로 후진 1,320mm

`BackToFinishZoneAndGameOver()`

마지막 블록을 B에 넣었다

오른쪽 290mm, 앞으로 950mm (CELL1 복귀 없음)

`FinishFromDropboxB()`

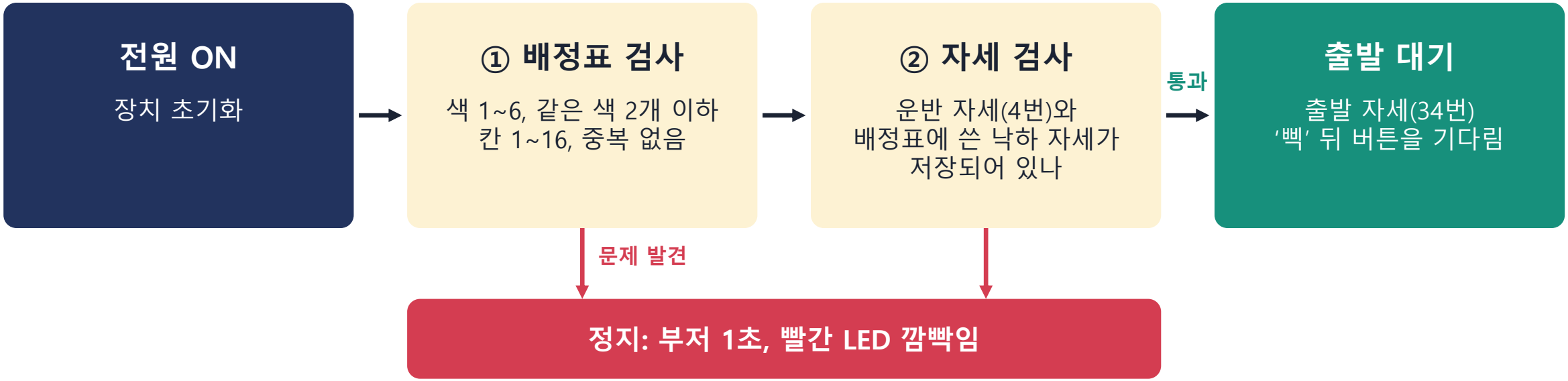
남은 블록을 끝내 못 찾았다

CELL1에서 왼쪽 124mm 이동 뒤 후진 1,320mm

`FinishFromCell1()`

종료하면 '삑' 두 번과 노란 LED 깜빡임으로 알립니다.

출발 버튼을 누르기 전에 로봇이 스스로 점검합니다



빨간 LED가 깜빡이면

시리얼 모니터를 봅니다. `Invalid missionAssignments table`은 배정표, `Missing pose ID n`은 n번 자세가 없다는 뜻입니다.

낙하 직전에도 한 번 더

낙하 자세가 없으면 그리퍼를 열지 않고 멈춥니다. 블록을 엉뚱한 곳에 떨어뜨리지 않습니다.

변경 6 · 안전 검사와 설정

더 빠르고 더 엄격해진 설정값

160mm/s

모든 직선 이동 속도

이전: 긴 이동 144, 옆 이동 108

80ms

층 자세를 잡고 촬영하기 전 대기

이전: 1,000ms

±2

앞쪽 PSD·카메라의 허용 오차

이전: ±3 (왼쪽 PSD는 3 그대로)

그 밖에 달라진 값	Mission2	Mission2B
장애물 앞까지 전진 거리	442mm	462mm
종료할 때 후진 거리	1,310mm	1,320mm
PSD 보정 속도의 상한	265	160
카메라 검출 간격 / 램프를 켠 뒤 대기	10ms / 300ms	30ms / 150ms
그리퍼를 닫는 값 (Devices.h)	930	955

거리와 PSD 값은 제공 코드에서 맞춘 값입니다. 내 로봇에서는 직접 재서 확인합니다.

Mission2B를 올리기 전에 준비할 것 4가지

1 모터 통신 속도 맞추기

`Motor.h` 의 통신 속도가 57,600에서 **1,000,000bps**로 바뀌었습니다. 모터 8개도 같은 속도로 바꿔야 통신됩니다.

도구 Tools / changeMotorBaudrate

2 새 팔 자세 저장하기

27번(한 번에 읽기), **34번**(출발), 그리고 배정표에 쓴 B 칸의 **19~26번** 낙하 자세가 필요합니다.

도구 Tools / poseMon, writeEEPROM

3 PSD 센서 4개 확인하기

오른쪽 PSD(A3)를 읽는 함수가 추가되었습니다. FL, FR, SL, SR 값이 모두 나오는지 봅니다.

도구 Tools / psdMon

4 직진과 옆 이동 다시 보기

이동 속도가 **160mm/s**로 바뀌어 바퀴별 비율도 새 값입니다. 곧게 가는지 확인합니다.

도구 Tools / motorAdjust

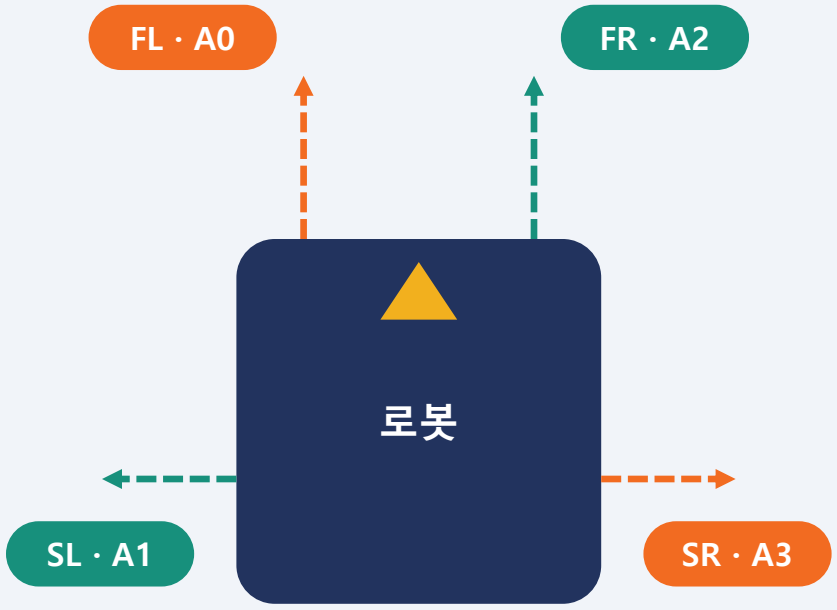
실습 준비

연습할 때 쓰는 스위치 5개

스위치	기본값	바꾸면 생기는 일
`REAL_WCRC_GAME_MODE`	`true`	false로 바꾸면 아래 연습용 기능을 쓸 수 있습니다.
`USE_SHORT_CUT_PASS`	`false`	true면 미션 읽기를 건너뛰고 적재함 앞에서 바로 시작합니다. 테스트용 배정표(B1~B4)를 씁니다.
`STOP_BEFORE_DROP`	`false`	true면 낙하 자세에서 멈추고 버튼을 기다립니다. 낙하 위치를 확인할 때 씁니다.
`PAUSE_AT_EACH_STEP`	`false`	true면 단계마다 버튼을 기다립니다. 경로를 하나씩 확인할 때 씁니다.
`RUN_TOP_ROW_SEARCH`	`true`	false면 맨 위층(L3)은 찾지 않습니다.

경기 전에는 반드시 `REAL_WCRC_GAME_MODE = true`, `USE_SHORT_CUT_PASS = false`

PSD 보정 함수 4개: 지금 쓰는 곳과 아직 안 쓰는 곳



청록: Mission2부터 쓰던 센서　주황: 새 함수가 쓰는 센서

함수	센서	맞추는 것	지금 부르는 곳
`PerformFrontMoveAlign`	FR	앞뒤 거리	장애물 앞, 각 CELL 앞
`PerformLeftMoveAlign`	SL	좌우 거리	미션지시 Zone, 탐색선
`PerformRightMoveAlign`	SR	좌우 거리	**아직 부르지 않음**
`PerformMoveRotateAlign`	FL + FR	앞뒤 거리와 비틀림	**아직 부르지 않음**

아래 두 함수는 **준비만 되어 있습니다.** 다음 시간에 필요한 자리에 직접 넣어 봅니다.

작은 오차도 끝까지: 허용 범위 밖인데 계산된 속도가 너무 작으면 최소 속도 10으로 움직입니다. Mission2는 그대로 멈췄습니다.

실습

내 로봇으로 확인해 봅시다

1 배정표 검사

같은 칸을 일부러 두 번 적고 올립니다. 빨간 LED로 멈추면 정상입니다.

2 미션 순서 읽기

시리얼 모니터의 `[MISSION] color IDs: 1 -> 3 -> …` 가 실제 순서와 같은지 봅니다.

3 B 경로 반복

`USE_SHORT_CUT_PASS = true` 로 시작해 회전 방향과 B 앞 도착 위치를 봅니다.

4 낙하 자세

`STOP_BEFORE_DROP = true` 로 A 한 칸, B 한 칸에서 멈춰 팔 위치를 봅니다.

5 마지막 CELL1

로봇이 CELL2로 넘어간 뒤 CELL1에 블록을 넣습니다. 돌아와서 집으면 정상입니다.

6 종료 위치

마지막 블록이 A일 때와 B일 때 도착한 자리를 각각 표시해 비교합니다.

기록할 때는 **코드에 적힌 값**, **내 로봇에서 잰 값**, **고쳐 넣은 값**을 나눠 적습니다.

정리

코드에서 찾아볼 함수

알고 싶은 것	찾을 이름	파일
배정표와 낙하 자세 번호	`missionAssignments`, `GetDropPoseId()`	Mission2B.ino
출발 전 검사	`AreMissionAssignmentsValid()`, `AreRequiredDropboxPosesPresent()`	Mission2B.ino
미션 순서 읽기	`ReadMissionFrame()`, `ReadMissionBlockSequence()`	Mission2B.ino
층 순서와 집기	`CamFindBlockAndPickup()`, `RunStorageLayerPose()`	Mission2B.ino
A/B 선택과 낙하	`DropAndReturn()`, `DropPickedBlockIntoDropbox()`	Mission2B.ino
B로 가기와 돌아오기	`GotoDropboxB()`, `ReturnFromDropboxBToCell1()`	Mission2B.ino
종료 경로	`FinishFromDropboxB()`, `FinishFromCell1()`	Mission2B.ino
제자리 회전	`RotateLeft90Degree()`, `RotateRight90Degree()`	Mobilebase.cpp

줄 번호는 코드를 고치면 바뀝니다. Ctrl+F로 이름을 검색하세요.

오늘의 정리

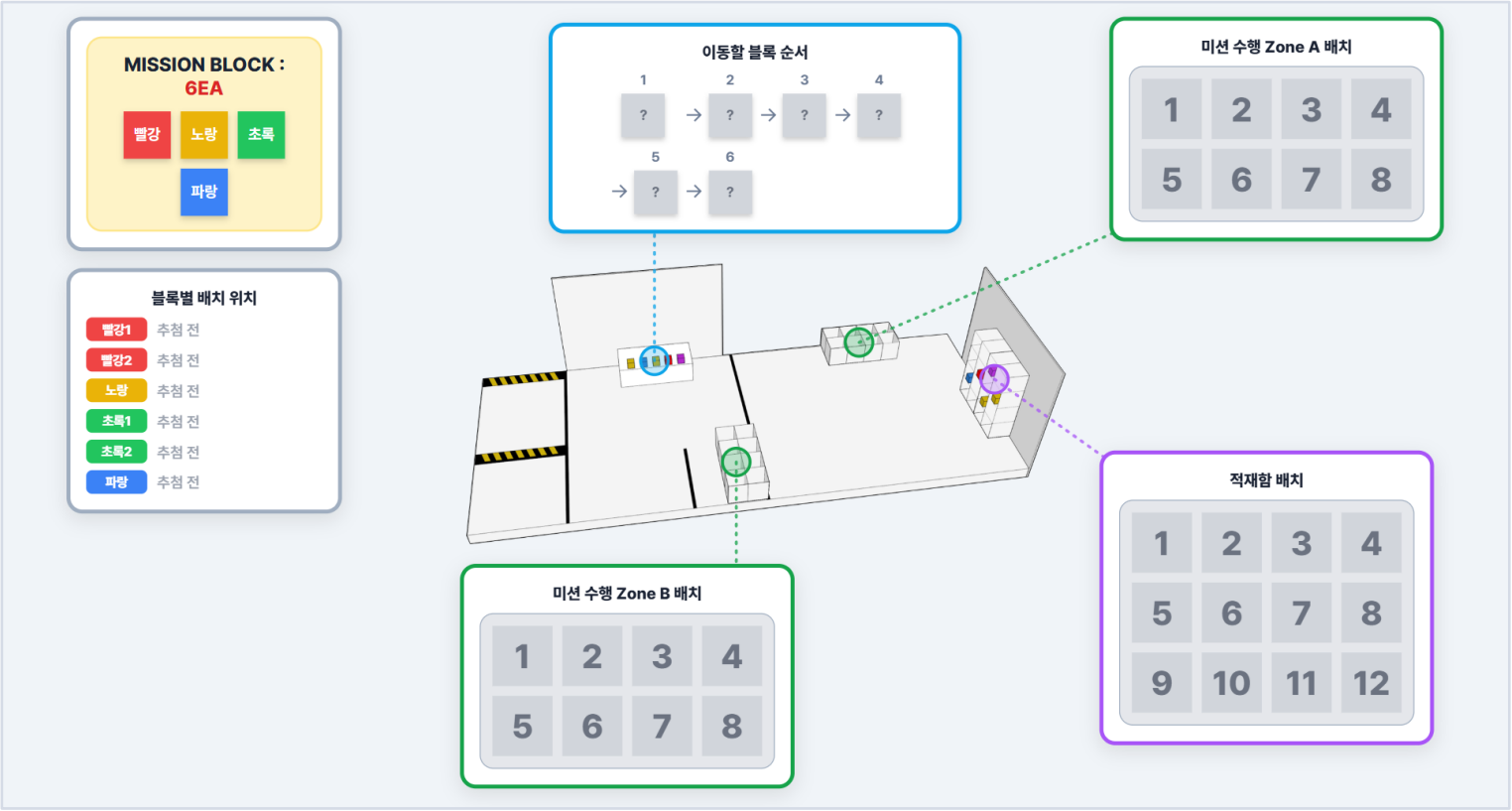
01 **목적지는 A와 B, 모두 16칸**
순서는 카메라가 읽고, 목적지는 배정표가 정합니다.

02 **탐색은 지그재그로 한 번 왕복**
팔 움직임과 대기 시간을 줄였고, 마지막 CELL1까지 확인합니다.

03 **B는 회전해서 가고, 그 자리에서 끝낼 수 있다**
마지막 블록을 B에 넣었다면 CELL1으로 돌아가지 않습니다.

다음 시간 코드에 TODO로 표시된 18곳을 직접 구현합니다

추첨 전: 사용할 블록의 색과 개수가 발표됩니다



추첨 전 화면. 블록의 색과 개수만 나와 있고, 목적지와 순서는 비어 있습니다.

언제 아침, 코딩 시간 전

화면에서 알 수 있는 것

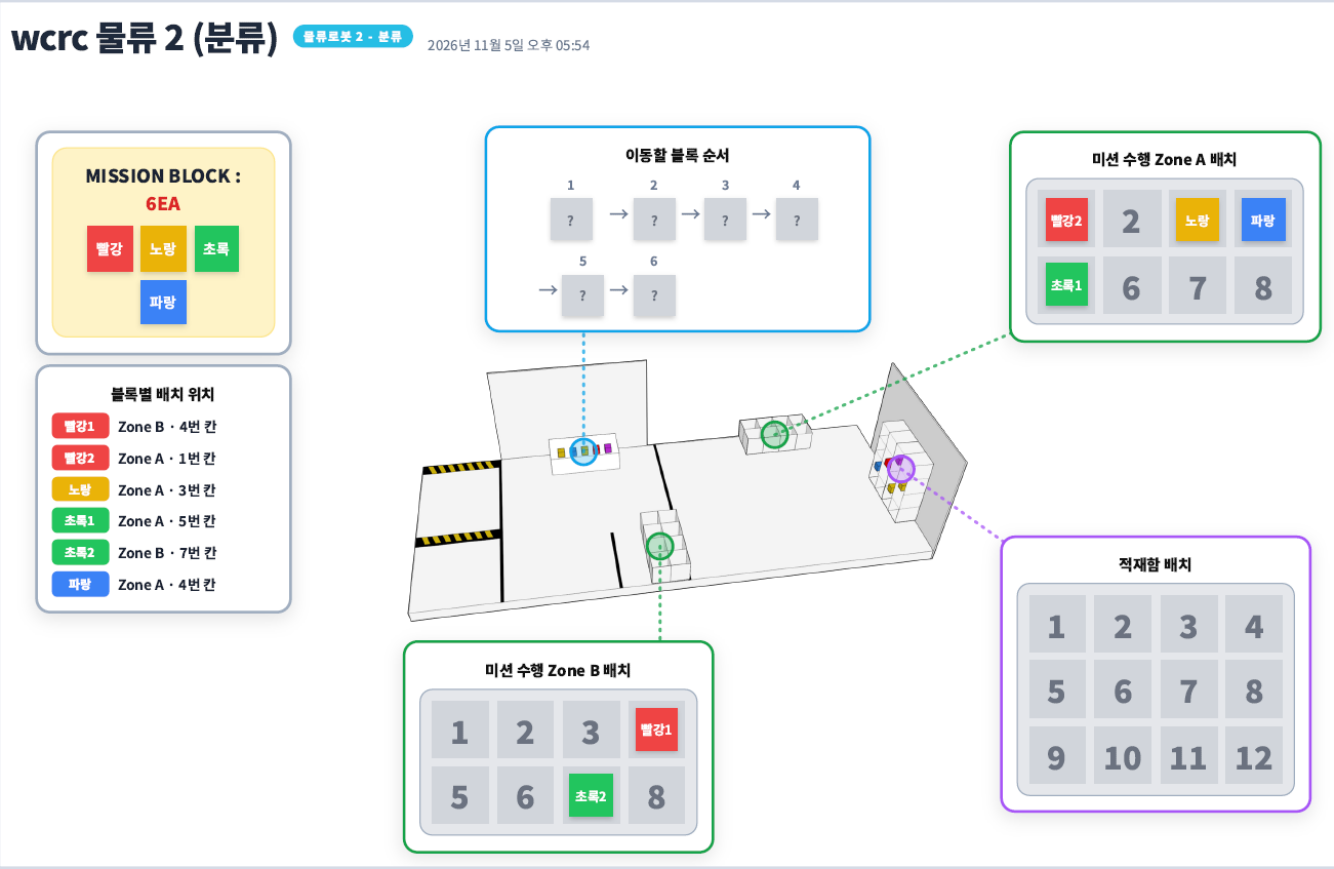
- 미션 블록은 모두 **6개**
- 빨강 2, 노랑 1, 초록 2, 파랑 1
- 같은 색은 빨강1, 빨강2처럼 번호로 구분
- 목적지와 순서는 아직 정해지지 않음

코드에서 할 일

배정표에 **색상 번호**부터 적어 둡니다.
빨강 1, 노랑 3, 초록 4, 파랑 5. 같은 색은 줄을 따로 씁니다.

➔ 다음의 1차 추첨 후 한 번에 수정 가능

1차 추첨: 블록마다 넣을 칸이 정해집니다



1차 추첨 화면. Zone A와 B의 칸에 블록이 배정되었습니다. 순서는 아직 '?'입니다.

언제 **추첨 전 발표 다음, 로봇 제출 전**

화면에서 알 수 있는 것

- 블록 6개가 들어갈 **Zone과 칸 번호**
- 예: 빨강1은 Zone B의 4번 칸
- 화면의 Zone A, B = 코드의 Dropbox A, B

코드에서 할 일

추첨 결과를 배정표 `missionAssignments[]` 에 적습니다. Zone A는 칸 번호 그대로, Zone B는 **8 + 칸 번호**로 적습니다. 적는 방법은 부록 4와 5에 있습니다.

➔ 마지막 페이지 코드 참고

2차 추첨: 옮길 순서와 적재함 배치가 정해집니다



2차 추첨 화면. 이동할 순서와 적재함 12칸의 배치가 채워졌습니다.

언제 로봇을 제출(임파운딩)한 뒤

화면에서 알 수 있는 것

- 옮길 순서: 초록, 초록, 파랑, 노랑, 빨강, 빨강
- 적재함 12칸 중 **10칸**에 블록이 놓임
- 미션에 없는 보라, 주황도 섞여 있음

코드에서 할 일

적어 넣을 수 없습니다. 제출한 뒤라서 코드를 고칠 수 없기 때문입니다. 순서는 카메라로 읽고(변경 2), 블록은 적재함을 탐색해 찾습니다(변경 3).

추첨 결과를 missionAssignments[ ]에 옮겨 적습니다

추첨 화면의 블록	배치 위치		색상 번호	낙하 칸		배정표에 적는 한 줄
빨강1	Zone B · 4번 칸	→	1	8+4	→	`{ 1, 8+4 },`
빨강2	Zone A · 1번 칸	→	1	1	→	`{ 1, 1 },`
노랑	Zone A · 3번 칸	→	3	3	→	`{ 3, 3 },`
초록1	Zone A · 5번 칸	→	4	5	→	`{ 4, 5 },`
초록2	Zone B · 7번 칸	→	4	8+7	→	`{ 4, 8+7 },`
파랑	Zone A · 4번 칸	→	5	4	→	`{ 5, 4 }`

색상 번호 빨강 1 · 주황 2 · 노랑 3 · 초록 4 · 파랑 5 · 보라 6

낙하 칸 Zone A는 칸 번호 그대로, Zone B는 8 + 칸 번호

Mission2B.ino의 missionAssignments[]에는 이렇게 입력합니다

```
#if USE_SHORT_CUT_PASS
  ...                          // 연습용 표: 그대로 둡니다
#else
  const MissionBlockAssignment missionAssignments[] = {
    { 1, 8+4 },    // 빨강1 → Zone B 4번
    { 1, 1   },    // 빨강2 → Zone A 1번
    { 3, 3   },    // 노랑  → Zone A 3번
    { 4, 5   },    // 초록1 → Zone A 5번
    { 4, 8+7 },    // 초록2 → Zone B 7번
    { 5, 4   }     // 파랑  → Zone A 4번
  };
#endif
```

2차 추첨의 순서는 입력하지 않습니다

로봇이 초록, 초록, 파랑, 노랑, 빨강, 빨강을 읽으면 **A5 → B7 → A4 → A3 → B4 → A1** 순서로 옮깁니다.

입력한 뒤 확인할 것

1. **줄 수가 블록 수와 같은가**
   이 예에서는 6줄입니다.

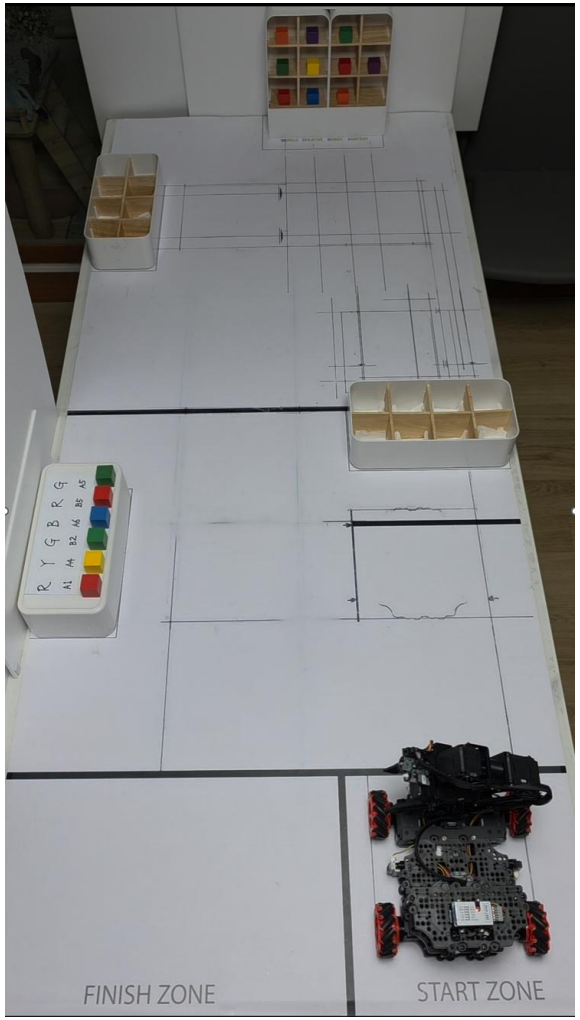
2. **같은 칸을 두 번 쓰지 않았는가**
   겹치면 출발 전 검사에서 빨간 LED로 멈춥니다.

3. **낙하 자세가 저장되어 있는가**
   A1, A3, A4, A5는 11, 13, 14, 15번. B4, B7은 22, 25번.

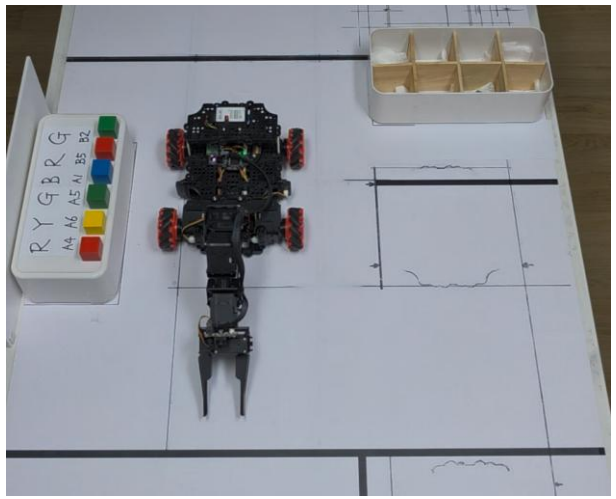
4. **같은 색을 번호 순서대로 적었는가**
   먼저 읽은 빨강이 표의 첫 번째 빨강 줄로 갑니다.

경기 때는 `USE_SHORT_CUT_PASS` 가 `false` 여야 이 표가 쓰입니다.

오전과 오후는 다른 경기장에 배정됩니다.







출발과 종료 시 확인할 사항

1 출발 시 로봇이 라인을 벗어나는가

로봇 팔을 포함하여 모든 부분이 START ZONE 안에 있어야 합니다.

2 출발과 종료 시 타이머 동작 기준

START/FINISH 라인에 레이저 빔 센서를 설치하여 라인 위의 약 2Cm 위치에서 출발/도착을 감지합니다.

3 미션 지시 블록은 같은 색상이 2개 중복 가능

동일 색상의 미션 블록에 대해서는 특히 첫번째와 두번째 블록의 낙하 박스 A와 B, 칸 구분이 중요함

4 적재함에는 미션 색상이 아닌 블록도 배치됨

12칸 적재함에는 미션 지시 블록의 색상이 아닌 다른 블록들도 임의로 배치될 수 있습니다.

5 드롭 박스 B에서 종료 주행 시 자세

드롭박스 B에서 마지막 블록을 낙하한 경우에는 로봇 팔을 길게 뻗어서 FINISH 라인으로 들어옵니다. (완료 후 밖으로 떨어지지 않도록 팔을 접어준다.)